

The impact of the Abilene Paradox on double-loop learning in an agile team

John McAvoy ^{*}, Tom Butler

Business Information Systems, University College Cork, Ireland

Available online 11 February 2007

Abstract

This paper presents a qualitative investigation of learning failures associated with the introduction of a new software development methodology by a project team. This paper illustrates that learning is more than the cognitive process of acquiring a new skill; learning also involves changes in behaviour and even beliefs. Extreme Programming (XP), like other software development methodologies, provides a set of values and guidelines as to how software should be developed. As these new values and guidelines involve behavioural changes, the study investigates the introduction of XP as a new learning experience. Researchers use the concepts of single and double-loop learning to illustrate how social actors learn to perform tasks effectively and to determine the best task to perform. The concept of triple-loop learning explains how this learning process can be ineffective, accordingly it is employed to examine why the introduction of XP was ineffective in the team studied. While XP should ideally foster double-loop learning, triple-loop learning can explain why this does not necessarily occur. Research illustrates how power factors influence learning among groups of individuals; this study focuses on one specific power factor – the power inherent in the desire to conform. The Abilene Paradox describes how groups can make ineffective decisions that are contrary to that which group members personally desire or believe. Ineffective decision-making occurs due to the desire to conform among group members; this was shown as the cause of ineffective learning in the software team studied. This desire to conform originated in how the project team cohered as a group, which was, in turn, influenced by the social values embraced by XP.

© 2007 Elsevier B.V. All rights reserved.

Keywords: XP adoption; Triple-loop learning; Social factors; Qualitative approach; Participant observation

1. Introduction

Extreme Programming (XP) is one of several software development methodologies that adopt an agile approach. Like other software development methodologies, it provides a set of values and guidelines as to how software should be developed. The adoption of XP into a project team is a learning experience like all other changes affecting team members. For many software developers and project managers, learning XP is simply a case of attending XP courses or reading one of the plethora of books on the subject. Many excellent courses exist and there are many informative and useful books to learn from, but learning is more than the cognitive process of acquiring a new skill. Learn-

ing involves a change in behaviour, even a change in beliefs. Researchers have described how learning is achieved and how it can fail. This study views the introduction of XP in one project team as a learning activity, and determines why this learning experience can be ineffectual and, ultimately, fail.

Several articles and books have detailed the optimum method of introducing Extreme Programming (XP) in a project team or teams. Some of these could be considered to be little more than descriptions of XP implementations, while others have rigorously researched the factors influencing the introduction of XP [32]. The deployment of a new software development methodology is an exercise in change – change in actions that lead to new behaviours – the Cutter IT Journal, for example, devoted two issues to an examination of XP and culture change. Such change is, in effect, an exercise in learning. The importance of

^{*} Corresponding author.

E-mail address: j.mcavoy@ucc.ie (J. McAvoy).

change in an XP project can be seen from the statement that “*change is viewed as the opportunity to learn and gain a competitive advantage rather than as a detriment to the process and its results*” [67, p. 40]. A review of papers on the changes brought about by the introduction of XP [56] concludes that there is a need to learn from the change. This paper echoes this observation and investigates the change brought about by the introduction of XP through the lens of the learning experiences of team members.

The remainder of this paper is structured as follows. Section 2 presents an overview of Extreme Programming (XP). Section 3 examines how learning occurs among individuals and groups. A relationship between the values of XP and these behaviours is posited and a claim is offered that the values inherent in XP practice should promote learning. The case study is then presented and conclusions offered; these illustrate that one of the core values of XP inhibited learning in practice; hence, weakening the above claim. Recommendations for future research are also considered.

2. The introduction of XP in a project team

XP is one of several agile software development methodologies; in fact, the founders of XP were among the originators of the Agile movement [66]. Agile is an umbrella term that describes methodologies for software development that adhere to four key values. Collectively, these values are known as the Agile Manifesto and are reported as [1,40,59]:

- Individuals and interactions over processes and tools.
- Working software over comprehensive documentation.
- Customer collaboration over contract negotiation.
- Responding to change over following a plan.

2.1. The principle and behaviours of XP

XP, as with the other agile methodologies, stresses a move away from traditional software development, which concentrates on process and documentation. XP underpins the need for an empowered software development team that is not overly burdened with processes, which inhibit the ability to adapt and change as the circumstances of the software project changes. Based on this principle, two of the key elements of XP are argued to be the development team (and its social interactions) and the acceptance and promotion of change within the project: these issues are now discussed.

XP places a much greater emphasis on people and teams than other methodologies [58,80] by concentrating on the social aspects of software development (as is evident from the Agile movement’s principles and values). Allied with this, its technical practices (or processes) also drive the social interaction in teams [45,69], striving for a sense of community within the team [27]. People issues are at the

core of the agile methodologies and this is evident in the levels of empowerment that team members possess [20]. In tandem with empowerment, a major defining characteristic of agile methods is the self-organisation of agile teams [27]. Previous research has specifically associated agile methods with empowerment and trust [18,76]; these are regarded as critical factors for an agile organisation. For example, teams are empowered to reject processes that do not add value [2]. “*Agile management is about letting everyone do what they are best at doing and trusting that they can do it right*” [76, p. 163]. Further advice is to “*build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done*” [55, p. 28]. This empowerment is one of the major reasons why developers like XP [80]; the logic being that it is empowerment, not monitoring, that motivates people [65]. It should be noted, however, that there are dissenting voices: these argue that agile methods are actually just another method of control within a project [16]. Such dissenting voices are in the minority though. Accepting the value of empowerment also means accepting that with empowerment comes ownership. Research has shown that collective ownership and collective responsibility are key features of teams adopting XP [69]; in fact, collective ownership is one of the twelve practices in XP [28,78,80]. User stories (simplified requirements) are an example of collective ownership in action, where the team has responsibility for delivery of all user stories, even in situations where individual team members accept responsibility for particular stories [28]. The move to empowerment of team members works in tandem with a new attitude (new for software development methodologies, that is) towards change. In agile methods, change is embraced as a necessity and a benefit, not something to be avoided. Of utmost importance for the successful change is the need for an agile team to honestly evaluate their work and to accept the need for changes to the software process [73]. Even the act of introducing XP to a project team is a change act. This is apparent as the adoption of an agile methodology, such as XP, will require often-significant cultural change [19,69].

2.2. Introducing XP as an experience in learning

Previous studies examine learning in the context of educating software engineers [60]. Software engineering students typically learn through participating in small well-bounded programming projects. The theory of situated learning posits that students must reflect on these projects, and the reality of the projects, to learn from them. Learning, nevertheless, is not restricted to education; learning also occurs within software development projects.

Argyris [3] describes learning as the detection and correction of errors. In a later work, Argyris [6] adds that learning also occurs when there is an initial match between our intentions and the consequences of our actions. It is clear from the literature that organisations and teams introducing XP do so with the ultimate objective of creat-

ing better software faster; this is achieved by following the values of the Agile Manifesto and the principles of XP. This is the stated aim, yet this goal may not translate into a reality. Often the decision to introduce XP is taken to correct problems with an existing, rigid methodology or even to introduce a methodology where none existed at all. As such, this is a learning process, as an error is detected with existing software development methods and the objective of introducing XP is to correct such errors. For learning to occur there must be a match between intentions – the desire to introduce and use XP – and the consequences – the successful introduction and subsequent use of XP. Before examining potential mismatches between the intention to introduce XP and the reality (the consequences) of the introduction, it is necessary to further examine the different forms of learning.

2.2.1. Single and double-loop learning

According to Argyris, there are two forms of learning – single- and double-loop [4–8]. Argyris uses the often-cited example of a thermostat to demonstrate single-loop learning: in this example, temperature is measured against a pre-defined value and the thermostat turns on or off the heat to maintain the temperature of the system at that value. In single-loop learning, the rationale behind the temperature choice are unchanged – for example, what lies behind a preference for a temperature setting of 28° as opposed to 32°? Double-loop learning occurs when strategies are examined, along with the assumptions that lie behind the choice of such strategies – that is individual norms and values are questioned leading to new theories-in-use [4,6,17,22,31]. Using the original analogy of a thermostat to demonstrate single-loop learning, the analogy of the thermostat is further developed by stating that if the thermostat could question why it is set to a particular temperature, or to examine why it worked a certain way, then the thermostat would be engaged in double-loop learning [7,8,17,29]. Double-loop learning not only questions the facts, it questions the logic behind the facts. Learning, be it single loop or double loop, should not be considered to reside exclusively in the domain of education. A review of extant research on traditional software development projects [15] found that organisations rarely learn from project post-mortems. The learning that occurs is, clearly, single-loop learning, whereas double-loop learning is rarely encountered. A similar point is made in a discussion of the management of software development projects, where managers are often so focused on finding solutions, that they fail to define the problems in the first place [47].

The values of the XP methodology appear to favour opportunities for double-loop learning, as opposed to single-loop learning – ideally this is a positive benefit of adopting XP. It should be noted that this can only be implied, as there is no extant research showing, or even looking for, a casual link between XP and double-loop learning. Despite this, existing literature on single and double-loop learning could be interpreted as implying a positive link. Single-loop

learning is predominant in bureaucratic organisations where employees are expected to keep to pre-defined roles and tasks [92]. As the agile methodologies advocate a move away from traditional bureaucratic management to one of empowerment and ownership, the opportunities for double-loop learning appear to be increased. Argyris argues that groups, and organisations, can achieve stability but, in doing so, limit learning [3]. Again, this should not be a major issue for XP, which embraces change as a core value. Change is accepted and actively encouraged – this is far removed from the stability described by Argyris as limiting learning. Furthermore, single-loop learning is relevant for individual goals as single-loop learning involves correcting errors, through individual inquiry, to maintain required norms. Team goals are more relevant to double-loop learning, as inquiry and feedback appears to be more dynamic in such settings [91]. The relevance of team goals to double-loop learning appears to match the importance of teams in the XP methodology. Rather than focusing on individual goals, the XP team is empowered and given ownership to achieve team goals. The relationship between XP and double-loop learning can be deduced from the argument that single-loop learning involves learning how to effectively perform a task, whereas double-loop learning involves determining which is the best task to perform and when change in performing that task is required [26,89]: Agile teams reflect on their work and make changes as required.

It has already been noted that for learning to occur there needs to be a match between individual intentions and their consequences [6]. Argyris and Schon illustrate the connection between intentions and consequences by relating learning and behaviour through their concepts of ‘theory in use’ (consequences) and ‘espoused theory’ (intentions) [4]. Espoused theories are those which individuals say they adopt in translating intentions to outcomes or consequences: for example, the belief in the benefits that accrue to an organisation by introducing XP to software development projects. ‘Theories-in-use’ reflect how individuals actually behave – what really guides action as opposed to what is stated publicly to so do. ‘Theories-in-use’, therefore, describe individual behaviours.

Much of human behaviour can be categorized as following a particular style ‘theory-in-use’ – Model I behaviour. This is operationalized in behaviour as winning (or not to be seen as losing or appearing incompetent), the suppression of negative feelings, and the importance of rationality, stability, and the avoidance of risk. This leads to the desire to protect the individual or the group, by appearing defensive and by reducing free choice and valid feedback. As a result, there is little genuine feedback on, or testing of, new ideas. An example of this is the concept of rationality, described by Argyris as a Model I behaviour, which is used to justify individual decision-making and behaviour and to discount the behaviour of others [22]. Argyris describes Model I behaviour as being based on single-loop learning [3,5–8]. By censoring what we say and hear, we prevent

learning. Model II behaviour moves away from the repressive censoring behaviours of Model I. The governing variables of Model II behaviour are valid information, free and informed choice, and commitment to decisions and actions. This behavioural model is based on double-loop learning and replaces winning and other Model I behaviours with open debate. To show how to move from Model I to Model II behaviour, Argyris uses workshops to confront the differences between ‘espoused theories’ and ‘theories-in-use’, thus tackling defence mechanisms directly [6,8,81].

XP strives for and requires Model II behaviours. The governing variables for Model II behaviour, and the double-loop learning it is based on, appear to be matched by the agile values of XP, as seen in Table 1.

Argyris highlights that Model I behaviour is the norm in most situations (this is not restricted to software development). Double-loop learning is thus not usually attained and ineffective learning is the norm. The failure to achieve Model II behaviour, therefore, is a failure to learn. As seen above, the use of the XP methodology appears to go against this negative norm of Model I behaviour by favouring and enhancing opportunities for double-loop learning. In an ideal situation, therefore, the use of XP should demonstrate double-loop learning seen through Model II behaviour. Although XP promotes, and appears to assist, Model II behaviour, not all XP projects are successful and could thus be considered as learning failures. The C3 project (Chrysler Comprehensive Compensation) was the first attempt at applying agile practices in a major project: this initiative was described as the father of XP, yet it was ultimately cancelled [19,80]. In some cases, therefore, there can be ineffective learning in XP projects. To examine why these failures (in XP and in learning) occur, it is necessary to examine why learning can be ineffective.

2.2.2. Triple-loop learning

To recapitulate: with single-loop learning norms and values remain unchanged, with double-loop learning norms and values are challenged and may undergo transformation. However, research illustrates that there are further

levels of learning above these. One of these is learning how to learn [64,75], which Bateson [14,85] refers to as deuterio learning, and Flood and Romm [39,71] refer to as triple-loop learning. “A person deuterio-learns when he learns about the process by which he changes his theory-in-use in response to experience” [75, p. 8]. To change our ‘theories-in-use’ requires a large degree of self-reflection and questioning. The highest levels of learning involve developing new structures and strategies to achieve double-loop learning [71]. Design, debate, and might–right issues (or power issues) are listed as the three loops of triple-loop learning [37,49,57]; Flood and Romm [39] describe the first-loop of their triple-loop learning as concerning “how”; that is, how organisations and processes are designed. An end is defined and the best means of arriving at this end is determined. If there is no structure in place to assist in these task oriented design decisions then there are inefficiencies. On the other extreme is the situation where there are too many structures in place, where the rules stifle choices. Flood and Romm liken this to a bureaucratic system. From this, it can be seen that a move away from traditional bureaucratic structures should aid learning within organisations [88]. As XP advocates a move away from bureaucratic structures, this appears to suggest that XP helps the learning process in the first loop. The second loop of Flood and Romm’s triple-loop learning concerns what decisions will be made. Whereas the first loop relates to how should we do things, the second loop relates to what should we do. Second-loop learning centres on debate and discussion on the choice of courses of action, and the freedom to debate these issues. While too little decision-making leads to inaction, the opposite of this does not necessarily lead to a better outcome. At the other extreme are superdecisions, which are based on assumptions that are unchallenged – for example, an expert expresses their opinion, and this is not debated by other interested decision-makers and stakeholders. This leads to forced decisions, usually keeping unchallenged assumptions while maintaining consensus. At both extremes, therefore, there is no power to influence decision-making.

Table 1
Double-loop and XP

Model II governing variables: double-loop learning	XP attributes
Valid information	“The agile manager fosters an environment where members of a team are able to make decisions and base them on the best information available” [76, p. 164].
Free and informed choice/open debate	Agile projects should have rules governing behaviour including [46]: • Everyone should have an equal voice • Everyone’s contribution is valuable • Attack issues not people • Keep privacy within the team • Respect each other and your differences • Everyone participates.
Commitment to decisions and actions	As everyone is involved though participative decision-making, the team will implement the decisions that are made [46]. A decision is implemented if it is accepted by those that implement it [86,87].

By combining these first two loops, which on their own are single-loop learning, we achieve double-loop learning. The third loop is a reflexive loop, where the important issue is how people effectively undertake the first two learning loops [39]. While the first two loops consider if things are done right and if the right things are being done, the third loop is a discourse about might and right. The power factors involved in learning are addressed by asking “is rightness the right of appeal to forms of science or presumed agreement, or the might of an appeal to the imperatives, that buttress a presumed conception?” [39, p. 172]. In effect, the question asked is if we are right because our position of power determines that what we say it right, or do we have a position of power because we are right. Politics and power relations have an impact here that must be considered, as they can have an impact on the first two loops. Do people have the power and freedom to make the choices required in the first two loops? Alternatively, do they have too much power, in which case decisions can also be flawed? Triple-loop learning occurs when we use all three loops of learning – by questioning if we are doing things right, if we are doing the right things, and if we make these decisions correctly.

2.2.3. *The Abilene Paradox*

If the attributes of XP match those of Model II behaviours and double loop learning, then a failure in the adoption of XP within a project can be considered to be a failure to learn. To determine the reason for a failure in the adoption of XP, it is necessary to examine the failure in the context of a failure to learn – what happens within the team, what social factors inhibit triple-loop learning, which in turn inhibit the adoption of XP? Flood and Romm [39] argue that power has an impact. While power may traditionally be seen as an individual asserting authority over a subordinate, the power of conformance is equally strong. As XP strongly advocates empowered teams with collective ownership, the cohesion of the team is vital in an XP team. This strong cohesion within a team can lead to power issues, where the pressure to conform becomes a concern. Writers on groupthink describe how this desire for cohesion in a team can ultimately lead to ineffective decision-making, where unanimity is of utmost importance, to the detriment of alternate or deviant views that may be actually correct [25,48,41,63]. Although groupthink is the most commonly described conformity issue in organisations and teams, there are alternative explanations. One such explanation, and a possible source of the impact of power in triple-loop learning, is the Abilene Paradox. The name derives from a trip to Abilene by Professor Jerry Harvey, with his family: the trip is described in [42–44,53,54,68]. On a hot day in July, Harvey and his family were content to be sitting on the back porch of his father-in-law. His father-in-law suggested a trip to Abilene for a meal. Harvey himself thought it a bad idea, but kept his view to himself when everyone else agreed. The trip and meal were ‘brutal’, to use Harvey’s

phrase. On their return, they gradually started to admit that it had been a bad idea. What was interesting, and the paradox of the theory’s name, is that the family members started to blame each other for the trip. Every member stated that they really hadn’t wanted to go. Even the initiator of the trip, his father-in-law, admitted that he never wanted to make the trip; he only proposed it as he thought the others would want to go. Harvey [42] described this as the Abilene Paradox, where people take actions that no one actually wants.

It is accepted that individuals can make decisions that ignore their own preferences because of rules, tradition, or the advice of others [59]; the term herd behaviour has been used to describe this [12,72]. Banerjee [12] describes an experiment where this behaviour is highlighted as taking the form of a sequence of decisions: for example, each decision maker examines the previous decisions before making his/her own. Banerjee found that individuals are overly influenced by the decision rules of others and fail to employ their own rules and information: this is described as herd behaviour. From this, it can be seen that the important influencers in a group decision are not necessarily the experts or the leaders. Power asymmetries aside, those who influence decisions of groups are those who voice their opinion first – it is usual for remainder of the group, the herd, to follow this lead. Asch describes an experiment that shows the impact that groups have on an individual’s decisions [10,11]. Asch’s experiment is also noted in [13,21,24,61,62]. Asch put together groups where one member is unaware that he/she is the one under study and other group members are playing an active role in the experiment. In conducting the experiment, group members not being studied deliberately give the same incorrect answers about the length of a line to see how this will affect the subject, whose reactions and decisions are being recorded. Asch reports that in general subjects conformed to the group’s decision, even though they knew it to be incorrect. Asch reports that seventy five percent of those studied agreed with the majority and gave incorrect answers. Asch uses this experiment to argue that groups tend to create conformity and consensus. An alternative view of this is that, although individuals can conform to the group’s wishes, in some cases, they may simply be obeying orders [52]. Whichever view one agrees with, it is accepted that group dynamics affect decision-making and agreement as, in many cases, individuals agree with decisions that they know to be incorrect. Harvey describes the Abilene Paradox as “the inability to manage agreement” [42,84], a phenomenon that has been found in groups within organisations [23,30,42–44,51,53,54]. Hence, it is clear that the Abilene Paradox is a potential power factor in triple-loop learning. This body of research identifies the symptoms of the Abilene Paradox as including situations where:

- Individual members privately agree on the correct decision to make; however, this is not shared with the group.

- Individual members privately agree on how the problem or situation being addressed can be resolved; again this is not shared with the group.
- Instead of communicating their views, members keep their views and reservations to themselves, agreeing with views they are opposed to.
- As the individuals have not presented their views and reservations, a collective decision is made that is actually contrary to the views of all members.
- Members feel frustration, even anger, at this and find someone, or some others, to blame.

This ineffective management of decisions has the potential to exert a negative influence on double-loop learning. Without the ability to correctly examine scenarios and to make decisions based on these scenarios, a team can not truly reflect and thus can not effectively learn. Flood and Romm [39] ask if individuals have the power to make informed decisions (and demonstrate learning). With the Abilene Paradox, individuals feel that they should not go against the grain, and thus do not make informed decisions. The power relationships within the team, as demonstrated by the Abilene Paradox, therefore, has the potential to adversely affect learning within cohesive teams such as an XP project team.

3. Research approach

The objective of this study was to examine how learning occurred, or did not occur, in the introduction of XP to a software development project. Accordingly the factors that aided learning and the factors which constrained learning were to be identified and explained. It should be noted that the identification of the Abilene Paradox emerged from the findings rather than being an *a priori* input into the study.

Learning, or the lack of learning, is often hidden, even from those involved. Individuals, for various reasons, are usually not able to determine why learning does not occur in their own case. This presents a problem for the use of many research approaches – for example, interviews may be considered redundant as a research tool if the interviewee is unaware of what they themselves are doing. To investigate the types of issues found in software development teams requires an approach where a team is examined in context and over an extended period. Examining social factors that inhibit learning involves determining factors that would be hidden from outsiders, and even from the individual team members themselves. Individuals can see the variance between another team member's 'espoused theories' versus their 'theories-in-use' more easily than they can identify their own 'espoused theories' and 'theories-in-use'; however, diplomacy usually prevents them from highlighting other's variances [3]. Surveys, questionnaires, and other such approaches are unsuitable as the respondent cannot, or may not wish to, identify the differences between 'espoused theories' and 'theories-in-use'. Cognitive dissonance implies that the team members would not be able

to examine and describe their own failures to learn. The original theory of cognitive dissonance was described by Festinger, in the 1950s. Here, individuals attempt to reduce cognitive discrepancies, even by changing their own opinion. A cognitive discrepancy occurs where facts appear to contradict beliefs or values. The individual attempts to remove the contradiction in order to maintain a logical consistency in their beliefs and opinions. In effect, individuals cannot recognise their own errors or faults; they will be ignored so as not to contradict with their beliefs about themselves: this theory is described and examined in various articles [9,35,36,74,82]. Weinberg took the concept of cognitive dissonance and specifically assigned it to software engineers [90]. Cognitive dissonance implies that questioning team members would be an ineffective method of determining, what is in effect, their failings. Even if the team were open to an analysis of their failings, they would be incapable of determining their failings themselves, and could not therefore describe them to a researcher.

To overcome such difficulties, a qualitative research approach using participant observation was adopted. Several studies have employed participant observation to conduct research into agile software development projects; the justification being that interpretative, in-depth case studies are the most appropriate method of investigating agile software development. For example, a qualitative approach involving participant observation was used by researchers to investigate the characteristics of an agile team and provided rich insights that could not be obtained by other research methods [70]. (It should be noted though, that the research described in [70] had a duration of only one week – this case had a duration of 8 months.) The trust gained by the researchers in such studies enabled them to examine factors that would otherwise have remained concealed. It is argued that it is the "taken for granted" that is worth observing in social contexts [77, p. 4], for it is the seemingly trivia of daily work that influence organisational outcomes as this everyday life constitutes reality [50]. Thus, a case study-based research approach using participation observation was chosen to research the phenomenon of interest.

A software development project team that had decided to introduce XP into a new development project was chosen for this study due the unique opportunity it afforded the researchers to observe the learning processes involved in the adoption and use of an agile method. This software development project team designed and developed a knowledge management system for a European government organisation. A team of seven developers and a project manager were involved in the project, the first phase of which lasted 8 months. Participant observation was chosen as the primary research technique to investigate the phenomenon of interest, as it is a particularly relevant approach when "the phenomenon is obscured from the view of outsiders" [50, p. 12]. The failure of translating 'espoused theories' or values into real action (i.e. the failure to learn) is one such phenomenon. Participant

observation of the development team occurred over an 8-month period, in what was a longitudinal research study. Both researchers participated as members of the team, and were therefore integrated into the group under study more or less continuously: for example, they participated in all team activities, including meetings, formal and informal discussions, and so on: such activities are argued to be vital in participant observation [34] as it “allows you to experience activities directly to get a feel of what events are like, and to record your own perceptions” [79, p. 51]. Furthermore, rather than being outsiders looking in, the researchers as team members took an active part in relevant development activities while also researching the phenomena. Detailed field notes were taken throughout and these were reflexively analysed and recorded by the researchers. The various themes, issues and group interactions were identified as the study progressed; and initial observations subsequently confirmed or disconfirmed. Thus, the study’s credibility was ensured due to prolonged engagement and persistent observation [33]: the researchers also peer debriefed each other, while member checks also occurred informally and formally at the end of the study when developers and team leader/project manager were interviewed.

4. Field observations

The objective of what was an innovative software development initiative was to create an enterprise-wide knowledge management system (KMS) for use in government and non-government organisations (NGOs); it was also designed so as to be customisable for use in commercial organisations. The project involved the creation of a full-featured core or kernel architecture that facilitates enterprise-wide knowledge management. The KMS system kernel provided a robust, open standards-based, reusable architecture that acted as the foundation for subsequent customised organisation-specific version, which was deployed in December 2005. The team was set up specifically for the sole purpose of undertaking the development of the KMS: however, the team comprised four team members from a previous project and four new members. As such, the team was an autonomous organisation with a single objective.

The investigation into the introduction of XP into the project was longitudinal, commencing with the decision to adopt XP and continuing through its introduction and actual use. The main focus of the study centred on the introduction of user stories, although other aspects of the introduction of XP were also observed, recorded and analysed. The focus on user stories was decided upon as at the core of XP is the use of user stories to underpin software development. Their importance can be seen in the argument that “*everything in XP revolves around user stories*” [80, p. 300]. User stories drive the work to be done and how to do it. Rather than relying on complex design documents, agile methods argue for the writing of customer

requirements in simple language. The stories should describe what is required of a part of the final software project. If the introduction of user stories failed, then XP (which revolves around stories) would also fail. Thus, the 8 months of observations was at its core an examination of the learning process inherent in the introduction of a new software development methodology – XP and user stories specifically.

The software development team’s core values aligned with those expressed in the Agile Manifesto. The project manager empowered the team to perform their individual tasks; however, ownership of the tasks rested with the team as a unit, as opposed to individual ownership. He ensured that the development team had access to all information from the clients and included team members in discussions with the client – nothing was hidden from the team. Because of this, the developers regarded the project manager as “one of the team” as opposed to an outsider in a command and control structure. As indicated four members of the development team had worked together on a previous software development project, including the project manager. In addition to the existing inter-personal relationships between team members, new friendships developed within the team and it became a cohesive group. This was evident in extracurricular activities, as the team regularly socialised together outside of work hours; accordingly, the work environment was extremely positive and the developers unanimously described the development site as a good place to work. The team evolved their own language, with terms and phrases unique to the group, and demonstrated an ability of ensuring that disagreements were accepted; but never got out of hand or hardened into enmity. When dealing with outsiders, the team tended to speak with one voice and defended each other, if their work was called into question. From the outset of the project, it was clear that cohesion was important to the team. Any team member, who went against the view of the majority, was usually brought back into line through informal talks between team members, which were universally amicable, with persuasion, rather than threats and heated arguments, being the norm.

The project manager suggested, and the team agreed with, the use of an agile software development approach for this new project. Prior to this, a more traditional document-oriented approach was attempted, although a specific methodology was not followed (it should be noted though that there are doubts if formal methodologies are actually completely adhered to anyway [38]). It was the failure to adhere to a formal methodology on a previous project that influenced the team’s desire to implement XP. In his capacity as a subject matter expert, one of the researchers provided an overview of XP to the team and he also suggested additional reading. Many of the team were already generally familiar with XP, without specifically having used it before on software projects.

The introduction of XP in this team involved more than a change in the software development process, it

also necessitates a change in working environment and values. However, during the study, the authors noted that the initial commitment to the use of XP decreased. When the project commenced, the team (both project manager and developers) agreed that a new process, or methodology, was required. It was felt that the previous project had problems in several areas, particularly in capturing and translating user requirements into the software solution; the project manager felt that these problems could be addressed or mitigated through the use of XP, which could also deal with the complexity and challenges associated with the development of an enterprise-wide system. This was a demonstration of single-loop learning – a realisation that “how” things were done needed to change. Initially, the entire project team enthusiastically welcomed the use of user stories. The team espoused their belief in the value of user stories and were determined to make use of them in the project. Correctly implemented, user stories are applied throughout the project as opposed to just the design phase, so this necessitated a longitudinal approach to the study. At the start of the development project there appeared to be full commitment to the use of user stories but, as time passed, the commitment effectively disappeared. Resistance to change would imply an initial negative reaction against the use of user stories. This was not the case, however, as following initial acceptance of user stories, there followed a slow dilution in commitment to their use. This dilution of commitment was manifested in sighs when user stories were mentioned, an unwillingness to take ownership of the stories, and the fact that they were not applied properly. Obstacles were placed in the way of their effective use, and it was team members that put into place these obstacles. The ‘espoused theory’ or beliefs were therefore not translated into ‘theories-in-use’, thus demonstrating a failure to learn.

The team members justified each obstacle they presented. Examples include:

- While XP advocates the use of user stories instead of traditional requirement documentation, the team argued that they still had to create some specification documents to “tie down” some architectural details.
- XP advocates the prioritisation of the user stories to determine when, and in what order, they will be coded. The team, lead by the project manager, argued that this went into too much detail and that they would prefer to have a general ranking of stories rather than prioritising them.
- Each story had to have enough work for, and no more than, a single development iteration (four weeks in this project). Several stories were of greater duration than four weeks, and the team argued that they simply couldn’t break the stories down to fit within the associated iteration. It is the opinion of the researchers that very little effort was made to make them fit – the developers simply stated they could not do it.
- Individual stories should have a risk assessment performed. The team did not do this; they simply stated that it was not worth the effort.

It should be noted that the progression, from initially espousing their belief in the value of user stories to their eventual abandonment, was a gradual one. It took place over approximately 6 months; the effect was similar to gradually “chipping away at” the use of user stories. What made this interesting was that each developer stated at the end of the project that they believed in the value of user stories. Interestingly, developers voiced their support individually, not as part of the group. One team member was instrumental in introducing user stories into the next project he was involved in, while a second developer recommended their use to a new company he joined.

The longitudinal aspect of the study showed how the team had gone full circle – espousing the benefit of user stories, failing to convert this into practice, and then back to espousing the benefit of user stories. This failure to translate espoused theories to theories-in-use (a failure to learn) appeared to oppose the Model II behaviour that XP appeared to promote. An examination of the variables of Model II behaviour showed how the team, at times, demonstrated behaviour that was the antithesis of Model II behaviour – Table 2. As we have seen above, the values of practices of XP appear to align well with Model II behaviour so this should not have occurred.

By the end of the project’s first major phase, it was clear that learning was not effective in the project team. While single-loop learning had been demonstrated (the team decided to change how they developed software), double-loop learning was not evident. Although XP appears to promote Model II behaviours, which demonstrate double-loop learning, this behaviour was not apparent in the team. What was not clear, however, was what was preventing effective learning, and the move to Model II behaviour? Triple-loop learning suggests that power issues prevent double-loop learning, but it was not initially clear what power issue was present in the XP project.

The importance of group relationships pointed to the possibility of groupthink having an impact. One of the main symptoms of groupthink is a pressure to conform to the group’s views [25,41,48,63]. This appeared to be relevant to this study as the group accepted initially the benefits of user stories. Over time the group failed to translate this into action. The group’s opinions also underwent change. As to an explanation as to why the change to, and application of, user stories was not effective, groupthink appeared to be having an effect, yet it does not explain why the group’s opinion changed from one of espousing benefit to one of lack of belief. The possibility of the existence of the Abilene Paradox (a dysfunctional power relationship within the team) was then investigated. The symptoms of the Abilene Paradox were compared to observations made in the longitudinal study: Table 3 performs a comparative analysis of symptoms and their

Table 2
Double-loop learning and observed behaviours

Model II/double-loop learning	Behaviour seen in team
Valid information	This behaviour was actually demonstrated, as all information was available to the team. At no time during the 8 months was information concealed. It could be argued, though, that by concentrating on team unity the team restricted the information they were willing to discuss. “Going against the grain” was frowned upon and actively discouraged.
Free and informed choice/open debate	At times, it was more important to display team unity than to have an open debate. “We must all sing from the same hymn sheet” was mentioned when a team member had a differing opinion. Team members specifically stated the importance of the team and team unity. Debates usually ended early in compromise rather than continuing discussions.
Commitment to decisions and actions	The team espoused commitment to user stories yet they were not translated into use.

manifestation in the actions and behaviours of the development team.

Four of the five symptoms of the Abilene Paradox were observed in the software development team studied (see Table 3). The final symptom, although it cannot be ruled out, was not observed directly and therefore the findings could not support its existence. It is partly confirmed, however, in that the developers complained regularly about requirements, and user stories are a manifestation of requirements. One developer specifically blamed the project manager for the failure, but others did not express this opinion. It is possible that the cohesion of the team, as required by XP, created the situation where team members did not openly criticise each other. The matching of the other symptoms to events in the case study shows how the Abilene Paradox ultimately led to a failure of learning. The group resisted the adoption of user stories yet the individual members of the group were not against their use.

The third-loop of the triple-loop learning theory describes how power factors inhibit learning; for this XP project team, the pressure to conform was the power factor observed at work in this team. While power may traditionally be seen as an individual asserting authority over a subordinate, the power of conformance is equally strong. Flood and Romm ask whether people have the power and freedom to make the choices required in the first two loops [39]. In the software development team studied

herein, the choices required to demonstrate effective learning were not made by either the project manager or individual team members; hence, the desire to conform prevented double-loop learning. This is important, as XP requires cohesive teams, empowered to take collective ownership and responsibility. This desire for cohesion did have positive benefits for the team, as was evident over the 8 months of the study, but it also led to ineffective learning. Nevertheless, it must be argued that the subsequent support for XP in subsequent projects had two team members advocate its adoption in those contexts.

The impact for those who practice XP is that its introduction can have unintended consequences and unforeseen negative side effects. To overcome the tendency of team members to conform, and the resultant unwillingness to offer contrary opinions, the authors conclude that it would be necessary to ensure a degree of intra-team conflict. While XP argues for cohesion, which appears to be the antithesis of conflict, it should still be possible to maintain cohesion while introducing low levels of positive conflict. The use of a devil’s advocate has been proposed as a solution to the Abilene Paradox [48,51], where a member of a group has the task of deliberately opposing or critiquing the group’s decision. For a cohesive XP team, where collective ownership of the project implies difficulty for team members to criticise what they themselves are part of, this may be problematic.

Table 3
An analysis of Abilene Paradox symptoms and observations in the case study

Abilene Paradox symptom	Case study observations
Members, as individuals, privately agree on the correct decision to make. This is not shared with the group.	Each developer stated the benefit of user stories, yet maligned them in group discussions.
Members, as individuals, privately agree on how the problem or situation being addressed can be resolved. This is not shared with the group.	Each developer stated how user stories should have been used, but individually – not in the group. Two team members, when later taking part in a different group, actually recommended the use of user stories.
Instead of communicating their views, members keep their views and reservations to themselves, agreeing with views they are opposed to.	Although privately agreeing with the use of user stories, no individual developer spoke up for them in the group
As the individuals have not presented their views and reservations, a collective decision is made that is actually contrary to the views of all members.	The resistance to user stories occurred collectively, yet it went against the individual opinions.
Members feel frustration, even anger, at this and find someone, or some people, to blame.	This is the only symptom that was not observed directly, although there is some anecdotal evidence of its existence.

A solution to this would involve the project manager taking on the role of devil's advocate. In this case, the project manager was part of the team, working day to day with the team, and socialising with the team outside of work. By adopting a role which is more removed from the XP team, and by questioning decisions and choices, as opposed to being part of the decision-making process, the Abilene Paradox can be controlled, if not removed completely. For XP teams to function correctly, with correct and suitable decision-making, an XP project manager must assume a role that is more external and remote to the team than that normally assumed. This is not to say that the XP project manager should not be considered part of the team (within and outside of working hours) but there should be a degree of separation and objectivity when decisions are made.

5. Recommendations for future research

This qualitative case study demonstrates how effective learning can be curtailed by the values inherent in XP and agile approaches. It therefore highlights a potential area of concern for those wishing to introduce an XP software development methodology. A potential flaw with qualitative studies is that they tend towards detailed examinations of specific issues or cases; consequently, those of a positivist bent argue that it is difficult to generalise from the findings of a single case study. The authors accept this, but it was not this study's aim to do so, rather it highlights areas of concern to practice, where matters of context are important, while also pointing towards factors that need to be included when making theoretical conjectures for empirical testing. On the issue of context, it must also be noted that team members and researchers were of the same nationality and cultural background (as is recommended for this particular method of research [50,83]). That said, this paper highlights a problem that has the potential to cause difficulties in other project teams. Nevertheless, it is unclear if the problem of a failure to learn in XP teams adopting agile approaches would affect cross-cultural initiatives. Clearly researchers from different cultures would be best placed to examine if the findings of this research can be transferred to teams operating in other cultural settings: for example, the dynamics and tendency to conform would be expected to be different across Asian, US, and European teams. A further potential flaw with the choice of participant observation is the possibility of bias and a lack of objectivity. This has to be accepted as a potential problem, although it can be mitigated in situations where researchers act in different roles in the team, or where one is internal and the other external. As in the case of the present study, if one researcher acts as project manager and another as a developer, this would allow both sides of the developer/management understandings to be visible. The observations and findings of this study were member checked and discussed to ensure that the viewpoint of management and

development were considered in an unbiased manner. Nevertheless, it is clear that participant observation will always tend towards a subjective view, but the longitudinal aspect of the study does provide sufficient data to challenge any *a priori* assumptions.

Further research is underway in three other teams introducing XP to software development projects that deal with different problem domains. Preliminary findings illustrate that some of the issues that arose in this case study have been manifested in other XP teams, yet the Abilene Paradox does not appear to provide an answer to, or explain, such observations. Hence, one must accept that the Abilene Paradox does not provide an all-encompassing explanation for the failure to learn in all XP projects. What is clear, however, is that the use of XP and the values it embraces does have the potential to affect the ability of a team to negotiate the gulf that separates stated beliefs and their application in practice (i.e. from 'espoused theories' to 'theories-in-use'). Can XP cause dysfunctional learning outcomes? A definite answer is only possible though further longitudinal case studies with researchers acting as participant observers. Hence, we argue that a quantitative approach would have been an inappropriate technique to uncover the issues that surfaced in this investigation. Surveys, interviews, or even experiments, would not have detected the underlying factors in the failure to learn by team members. This is not an argument against quantitative approaches, which are of considerable value in other research settings and on other phenomena. What the authors propose, however, is suitable methods need to be employed in order to discover often latent power-related factors. If the power-related factors that influence the software process remain hidden even from those demonstrating the existence of such factors, then there is a requirement for investigation techniques that place the researcher in a position to observe subtle changes of attitudes and behaviours of team members over a considerable period of time. The qualitative approach of participant observation meets this requirement.

References

- [1] P. Abrahamsson, O. Salo, J. Ronkainen, J. Warsta, Agile Software Development Methods. Review and Analysis, VTT Publications, Finland, 2002.
- [2] K. Aguanno, Managing Agile Projects, Multimedia Publications, Ontario, Canada, 2005.
- [3] C. Argyris, Single-loop and double-loop models in research on decision making, *Administrative Science Quarterly* 21 (3) (1976) 363–375.
- [4] C. Argyris, S. Schon, *Organizational Learning: A Theory of Action Perspective*, Addison-Wesley, MA, USA, 1978.
- [5] C. Argyris, Good communication that blocks learning, *Harvard Business Review* 72 (4) (1994) 77–85.
- [6] C. Argyris, Action science and organizational learning, *Journal of Managerial Psychology* 10 (6) (1995) 20–26.
- [7] C. Argyris, Double loop learning in organizations, *Harvard Business Review* 55 (5) (1997) 115–125.
- [8] C. Argyris, Double loop learning, teaching, and research, *Academy of Management. Learning and Education* 1 (2) (2002) 206–218.

- [9] E. Aronson, J. Mills, Effect of severity of initiation on liking for a group, in: D. Cartwright, A. Zander (Eds.), *Group Dynamics*, Harper & Row Publishers, NY, USA, 1968, pp. 119–124.
- [10] S. Asch, *Social Psychology*, Prentice-Hall, NJ, USA, 1952.
- [11] S. Asch, Opinions and social pressure, *Scientific American* 193 (5) (1955) 31–35.
- [12] A. Banerjee, A simple model of herd behaviour, *The Quarterly Journal of Economics* 52 (3) (1992) 797–817.
- [13] R. Baron, N. Kerr, N. Miller, *Group Process, Group Decision, Group Action*, Open University Press, Buckingham, UK, 1999.
- [14] G. Bateson, *Steps to an Ecology of Mind*, Ballantine Books, NY, USA, 1972.
- [15] P. Benyon-Davies, I. Owens, M. Williams, Information systems evaluation and the information systems development process, *Journal of Enterprise Information Management* 17 (4) (2004) 276–282.
- [16] D. Birmingham, *Software Development on a Leash*, Springer-Verlag, NY, USA, 2002.
- [17] D. Blackman, J. Connelly, S. Henderson, Does double loop learning create reliable knowledge? *The Learning Organization* 11 (1) (2004) 11–27.
- [18] B. Boehm, R. Turner, Rebalancing your organizations agility and discipline, in: F. Maurer, D. Wells (Eds.), *Extreme Programming and Agile Methods – Xp/Agile Universe 2003*. New Orleans, Springer-Verlag, Berlin, Germany, 2003, pp. 1–8.
- [19] B. Boehm, R. Turner, *Balancing Agility and Discipline*, Pearson Education, MA, USA, 2004.
- [20] B. Boehm, R. Turner, Management challenges to implementing Agile processes in traditional development organizations, *IEEE Software* 22 (5) (2005) 30–39.
- [21] D. Boje, J. Murnighan, Group confidence pressures in iterative decisions, *Management Science* 28 (10) (1982) 1187–1196.
- [22] R. Bokeno, The work of Chris Argyris as critical organization practice, *Journal of Organizational Change Management* 16 (6) (2003) 633–649.
- [23] A. Carlisle, An Abilene defense: commentary two, *Organizational Dynamics* 17 (2) (2001) 40–43.
- [24] D. Cartwright, A. Zander, Pressures to uniformity in groups, in: D. Cartwright, A. Zander (Eds.), *Group Dynamics*, Harper & Row Publishers, NY, USA, 1968, pp. 139–151.
- [25] R. Cartwright, *Mastering Team Leadership*, Palgrave Macmillan, Wales, 2002.
- [26] S. Clegg, M. Kornberger, T. Pitsis, *Managing and Organizations: An Introduction to Theory and Practice*, Sage Publications, London, UK, 2005.
- [27] A. Cockburn, J. Highsmith, Agile software development: the people factor, *IEEE Computer* 34 (11) (2001) 131–133.
- [28] M. Cohn, *User Stories Applied for Agile Software Development*, Addison-Wesley, MA, USA, 2004.
- [29] R. Denhardt, J. Denhardt, M. Aristigeuta, *Managing Behaviour in Public and Nonprofit Organizations*, Sage Publications, CA, USA, 2002.
- [30] J. Dumville, The Abilene paradox, *Journal of Management Engineering* 15 (1) (1999) 6.
- [31] M. Easterby-Smith, M. Lyles, Re-reading organizational learning: selective memory, forgetting, and adaptation, *Academy of Management Executive* 17 (2) (2003) 51–55.
- [32] J. Erickson, K. Lyytinen, K. Siau, Agile modelling, Agile software development, and Extreme Programming: the state of research, *Journal of Database Management* 16 (4) (2005) 88–100.
- [33] D. Erlandson, E. Harris, B. Skipper, S. Allen, *Doing Naturalistic Inquiry: A Guide to Methods*, Sage Publications, London, UK, 1993.
- [34] P. Ezey, Integration and its challenges in participant observation, *Qualitative Research* 3 (2) (2003) 191–205.
- [35] L. Festinger, J. Carlsmith, Cognitive consequences of forced compliance, in: E. Coats, R. Feldman (Eds.), *Classic and Contemporary Readings in Social Psychology*, third ed., Pearson Education, NJ, USA, 1959, pp. 194–203.
- [36] L. Festinger, E. Aronson, Arousal and reduction of dissonance in social contexts, in: D. Cartwright, A. Zander (Eds.), *Group Dynamics*, Harper & Row Publishers, NY, USA, 1968, pp. 125–136.
- [37] P. Finnegan, R. Galliers, P. Powell, Applying triple loop learning to planning electronic trading systems, *Information Technology and People* 16 (4) (2003) 461–483.
- [38] B. Fitzgerald, The use of systems development methodologies in practice: a field study, *Information Systems Journal* 7 (3) (1997) 210–212.
- [39] R. Flood, N. Romm, *Diversity Management: Triple Loop Learning*, John Wiley and Sons, London, UK, 1996.
- [40] M. Fowler, J. Highsmith, The agile manifesto, *Software Development* 9 (8) (2001) 28–32.
- [41] E. Griffin, *A First Look at Communication Theory*, McGraw-Hill, NY, USA, 1997.
- [42] J. Harvey, The Abilene paradox: the management of agreement, *Organizational Dynamics* 3 (1) (1974) 63–80.
- [43] J. Harvey, Abilene revisited: an epilogue, *Organizational Dynamics* 17 (2) (2001) 35–37.
- [44] J. Harvey, R. Buckley, M. Novicevic, J. Halbesleben, The Abilene paradox after thirty years: a global perspective, *Organizational Dynamics* 33 (2) (2004) 215–226.
- [45] O. Hazzan, J. Tomayko, Human aspects of software engineering, in: *Proceedings of Extreme Programming and Agile Processes in Software Engineering*. 5th International Conference, Springer-Verlag, Germany, 2004, pp. 303–311.
- [46] J. Highsmith, *Agile Project Management*, Pearson Education, MA, USA, 2004.
- [47] W. Humphrey, *Managing the Software Process*, Addison-Wesley, NY, USA, 1989.
- [48] I. Janis, *Victims of Groupthink*, Houghton Mifflin Company, MA, USA, 1972.
- [49] P. Jarvinen, Potential research methods for studying virtual workplace, in: P. Kackson, R. Suomi (Eds.), *Ebusiness and Workplace Design*, Routledge, London, UK, 2002, pp. 155–172.
- [50] D. Jorgensen, *Participant Observation: A Methodology for Human Studies*, Sage Publications, CA, USA, 1989.
- [51] R. Kanter, An Abilene defense: commentary one, *Organizational Dynamics* 17 (2) (2001) 37–40.
- [52] M. Kent, Conformity, in: A. Hare, H. Blumberg, M. Davies, M. Kent (Eds.), *Small Group Research: A Handbook*, Ablex Publishing, NJ, USA, 1994, pp. 107–137.
- [53] Y. Kim, A comparative study of the ‘Abilene paradox’ and ‘groupthink’, *Public Administration Quarterly* 25 (2) (2001) 168–187.
- [54] A. LaPlante, 90s style brainstorming, *Forbes* 150 (10) (1993) 44–51.
- [55] C. Larman, V. Basili, Iterative and incremental development: a brief history, *IEEE Computer* 36 (6) (2003) 47–58.
- [56] T. Larsen, The phenomenon of diffusion, in: M. Ardis, B. Marcolin (Eds.), *Diffusing Software Product and Process Innovations*, Kluwer Academic Publishers, MA, USA, 2001, pp. 35–50.
- [57] B. Lehaney, S. Clarke, E. Coakes, G. Jack, *Beyond Knowledge Management*, Idea Group Publishing, PA, USA, 2004.
- [58] M. Lindvall, V. Basili, B. Boehm, P. Costa, K. Dangle, F. Shull, R. Tesoriero, L. Williams, M. Zelkowitz, Empirical findings in agile methods, in: *Proceedings of Extreme Programming and Agile methods – XP/Agile Universe*. Chicago, Springer-Verlag, NY, USA, 2002, pp. 197–207.
- [59] J. March, Rationality, ambiguity, and the engineering of choice, in: D. Bell, H. Raiffa, A. Tversky (Eds.), *Decision Making. Descriptive, Normative, and Prescriptive Interactions*, Cambridge University Press, Cambridge, UK, 1989, pp. 33–57.
- [60] L. Mathiassen, S. Purao, Educating reflective systems developers, *Information Systems Journal* 12 (2) (2002) 81–102.
- [61] C. McCauley, Group dynamics in Janis’s theory of groupthink: backward and forward, *Organizational Behaviour and Human Decision Processes* 73 (2/3) (1998) 142–162.

- [62] S. Milgram, Group pressure and action against a person, in: D. Taylor (Ed.), *Small Groups*, Markham Publishing Company, IL, USA, 1971, pp. 157–170.
- [63] G. Moorhead, C. Neck, M. West, The tendency towards defective decision making with self-managing teams: the relevance of group-think, *Organizational Behaviour and Human Decision Process* 73 (2/3) (1998) 327–351.
- [64] G. Morgan, *Creative Organization Theory: A Resourcebook*, Sage Publication, CA, USA, 1989.
- [65] M. Poppendieck, T. Poppendieck, *Lean Software Development: An Agile Toolkit for Software Development Managers*, Addison-Wesley, NY, USA, 2003.
- [66] J. Rainwater, *Herding Cats: A Primer for Programmers Who Lead Programmers*, Springer-Werlag, NY, USA, 2002.
- [67] D. Riehle, A comparison of the value systems of Adaptive Software Development and Extreme Programming: how methodologies may learn from each other, in: G. Succi, M. Marchesi (Eds.), *Extreme Programming Explained*, Addison-Wesley, Boston, USA, 2001, pp. 35–50.
- [68] H. Robbins, M. Finley, *Why Teams Don't Work*, Orion Publishing, London, UK, 1998.
- [69] H. Robinson, H. Sharp, XP Culture: why the twelve practices both are and are not the most significant thing, in: *Proceedings of Agile Development Conference*, Salt Lake City, IEEE Computer Society, 2003, pp. 12–21.
- [70] H. Robinson, H. Sharp, The social side of technical practices, in: *Proceedings of 6th International Conference on Extreme Programming and Agile Processes in Software Engineering (XP2005)*, Springer-Verlag, Sheffield, UK, 2005, pp. 100–108.
- [71] N. Romme, A. Van Witteloostuijn, Circular organizing and triple loop learning, *Journal of Organizational Change Management* 13 (5) (1999) 439–453.
- [72] D. Scharfstein, J. Stein, Herd behaviour and investment, *The American Economic Review* 80 (3) (1990) 465–479.
- [73] B. Schatz, I. Abdelshafi, Primavera gets Agile: a successful transition to Agile development, *IEEE Software* 22 (3) (2005) 36–42.
- [74] T. Schelling, The mind as a consuming organ, in: D. Bell, H. Raiffa, A. Tversky (Eds.), *Decision making. Descriptive, Normative, and Prescriptive Interactions*, Cambridge University Press, Cambridge, UK, 1989, pp. 343–357.
- [75] D. Schon, Deutero-learning in organizations: learning for increased effectiveness, *Organizational Dynamics* 4 (19) (1975) 2–16.
- [76] P. Schuh, *Integrating Agile Development in the Real World*, Delmar Thomson Learning, NY, USA, 2004.
- [77] H. Schwartzman, *Ethnography in Organizations*, Sage Publications, CA, USA, 1993.
- [78] H. Sharp, H. Robinson, An ethnographic study of XP practice, *Empirical Software Engineering* 9 (4) (2004) 353–375.
- [79] J. Spradley, *Participant Observation*, Holt, Rinehard, and Winston, NY, USA, 1980.
- [80] M. Stephens, D. Rosenberg, *Extreme Programming Refactored: The Case Against XP*, Springer-Verlag, Heidelberg, Germany, 2003.
- [81] P. Sun, J. Scott, Exploring the divide – organizational learning and the learning organization, *The Learning Organization* 10 (4) (2003) 202–215.
- [82] D. Taylor, Social comparison, in: D. Taylor (Ed.), *Small Groups*, Markham Publishing Company, IL, USA, 1971, pp. 86–87.
- [83] B. Tedlock, Ethnography and ethnographic presentation, in: N. Denzin, Y. Lincoln (Eds.), *Handbook of Qualitative Research*, second ed., Sage Publications, CA, USA, 2000, pp. 455–486.
- [84] F. Thomas, Leadership, the development cycle, maturity & power, in: *Proceedings of 18th Annual ACM SIGUCCS Conference on User Services*, ACM Press, Ohio, USA, 1990, pp. 399–402.
- [85] H. Volberda, *Building the Flexible Firm: How to Remain Competitive*, Oxford University Press, Oxford, UK, 1998.
- [86] C. Von Bergen, R. Kirk, Groupthink: when too many heads spoil the decision, *Management Review* 67 (3) (1978) 44–49.
- [87] V. Vroom, A normative model of managerial decision-making, in: D. Pugh (Ed.), *Organization Theory*, third ed., Penguin Books, London, UK, 1974, pp. 309–328.
- [88] C. Wang, C. Ahmed, Organizational learning: a critical review, *The Learning Organisation* 10 (1) (2003) 8–17.
- [89] R. Wardhaugh, A. Shani, P. Docherty, *Learning by Design*, Blackwell Publishing, Oxford, UK, 2003.
- [90] G. Weinberg, *The Psychology of Computer Programming*, Van Nostrand Reinhold Co., NY, USA, 1971.
- [91] R. Yeo, From individual to team learning, *Team Performance Management: An International Journal* 8 (7/8) (2002) 157–170.
- [92] L. Yeomans, Does reflective practice have relevance for innovation in public relations? *Journal of Communication Management* 5 (1) (2000) 72–81.